

Presentamos Google AI Edge Portal (<https://ai.google.dev/edge/ai-edge-portal?hl=es-419>): Compara la IA de Edge a gran escala. [Regístrate](#) (<https://docs.google.com/forms/d/e/1FAIpQLSfTcGPycQve8TLAsfH46pBIXBZe9FrgJAClwbF7DeL1LgVn4Q/vi ewform?hl=es-419>) para solicitar acceso durante la vista previa privada.

translated by **Google**

Se usó la [API de Cloud Translation](https://cloud.google.com/translate/?hl=es-419) (<https://cloud.google.com/translate/?hl=es-419>) para traducir esta página.

[Switch to English](#)

Guía de inferencia de LLM para iOS

Nota: El uso de la API de inferencia de LLM de MediaPipe está sujeto a la [Política de Uso Prohibido de IA Generativas](https://policies.google.com/terms/generative-ai/use-policy?hl=es-419) (<https://policies.google.com/terms/generative-ai/use-policy?hl=es-419>).

La API de LLM Inference te permite ejecutar modelos grandes de lenguaje (LLM) completamente en el dispositivo para aplicaciones para iOS, que puedes usar para realizar una amplia variedad de tareas, como generar texto, recuperar información en forma de lenguaje natural y resumir documentos. La tarea proporciona compatibilidad integrada con varios modelos grandes de lenguaje de texto a texto, de modo que puedas aplicar los modelos de IA generativa más recientes en el dispositivo a tus apps para iOS.

Para agregar rápidamente la API de LLM Inference a tu aplicación para iOS, sigue la [Guía de inicio rápido](#) (#quickstart). Para ver un ejemplo básico de una aplicación para iOS que ejecuta la API de LLM Inference, consulta la [aplicación de ejemplo](#) (#sample-application). Para obtener una explicación más detallada del funcionamiento de la API de LLM Inference, consulta las secciones [opciones de configuración](#) (#configuration-options), [conversión de modelos](#) (#model-conversion) y [ajuste de LoRA](#) (#lora-customization).

Puedes ver esta tarea en acción con la [demo de MediaPipe Studio](https://mediapipe-studio.webapps.google.com/studio/demo/llm_inference?hl=es-419) (https://mediapipe-studio.webapps.google.com/studio/demo/llm_inference?hl=es-419). Para obtener más información sobre las funciones, los modelos y las opciones de configuración de esta tarea, consulta la [descripción general](#) (https://ai.google.dev/edge/mediapipe/solutions/genai/llm_inference/index?hl=es-419).

Guía de inicio rápido

Sigue estos pasos para agregar la API de LLM Inference a tu aplicación para iOS. La API de LLM Inference usa la biblioteca `MediaPipeTasksGenai`, que se debe instalar con CocoaPods. La biblioteca es compatible con apps de Swift y Objective-C, y no requiere ninguna configuración adicional específica del lenguaje.

Para obtener instrucciones para instalar CocoaPods en macOS, consulta la [guía de instalación de CocoaPods](https://guides.CocoaPods.org/using/getting-started.html) (<https://guides.CocoaPods.org/using/getting-started.html>). Para obtener instrucciones sobre cómo crear un `Podfile` con los pods necesarios para tu app, consulta [Cómo usar CocoaPods](https://guides.CocoaPods.org/using/using-cocoapods.html) (<https://guides.CocoaPods.org/using/using-cocoapods.html>).

Cómo agregar dependencias

Agrega el pod `MediaPipeTasksGenai` en `Podfile` con el siguiente código:

```
target 'MyLlmInferenceApp' do
  use_frameworks!
  pod 'MediaPipeTasksGenAI'
  pod 'MediaPipeTasksGenAIC'
end
```

Si tu app incluye destinos de pruebas de unidades, consulta la [Guía de configuración para iOS](https://ai.google.dev/mediapipe/solutions/setup_ios?hl=es-419) (https://ai.google.dev/mediapipe/solutions/setup_ios?hl=es-419) para obtener información adicional sobre cómo configurar tu `Podfile`.

Descarga un modelo

Descarga Gemma-2 2B en un formato cuantificado de 8 bits desde [Modelos de Kaggle](https://www.kaggle.com/models/google/gemma-2/tfLite/gemma2-2b-it-gpu-int8T) (<https://www.kaggle.com/models/google/gemma-2/tfLite/gemma2-2b-it-gpu-int8T>). Para obtener más información sobre los modelos disponibles, consulta la [documentación de los modelos](https://ai.google.dev/edge/mediapipe/solutions/genai/llm_inference/index?hl=es-419#models) (https://ai.google.dev/edge/mediapipe/solutions/genai/llm_inference/index?hl=es-419#models).

Agrega el modelo al directorio de tu proyecto con Xcode. Si deseas obtener instrucciones para añadir archivos a tu proyecto de Xcode, consulta [Cómo administrar archivos y carpetas en tu proyecto de Xcode](https://developer.apple.com/documentation/xcode/managing-files-and-folders-in-your-xcode-project) (<https://developer.apple.com/documentation/xcode/managing-files-and-folders-in-your-xcode-project>).

Inicializa la tarea

Inicializa la tarea con las opciones de configuración básicas:

```
import MediaPipeTasksGenai

let modelPath = Bundle.main.path(forResource: "model",
                                  ofType: "bin")

let options = LlmInferenceOptions()
options.baseOptions.modelPath = modelPath
options.maxTokens = 1000
options.topk = 40
options.temperature = 0.8
options.randomSeed = 101

let llmInference = try LlmInference(options: options)
```

Nota: Si se establece la temperatura o Top-K sin establecer también la semilla aleatoria, no habrá variabilidad en la respuesta.

Ejecuta la tarea

Usa el método `generateResponse(inputText:)` para generar una respuesta de texto. Esto genera una sola respuesta generada.

```
let result = try LlmInference.generateResponse(inputText: inputPrompt)
```

Para transmitir la respuesta, usa el método `generateResponseAsync(inputText:)`.

```
let resultStream = LlmInference.generateResponseAsync(inputText: inputPrompt)

do {
    for try await partialResult in resultStream {
        print("\(partialResult)")
    }
    print("Done")
}
catch {
    print("Response error: '\(error)'")
}
```

Nota: La tarea bloquea el subproceso actual hasta que termina de ejecutar la inferencia en el texto. Para evitar bloquear el subproceso actual, ejecuta el procesamiento en un subproceso en segundo plano con los frameworks [Dispatch](https://developer.apple.com/documentation/dispatch) (<https://developer.apple.com/documentation/dispatch>) o [NSOperation](https://developer.apple.com/documentation/foundation/nsoperation) (<https://developer.apple.com/documentation/foundation/nsoperation>) de iOS.

Aplicación de ejemplo

La aplicación de ejemplo es un ejemplo de una app de generación de texto básica para iOS que usa la API de inferencia de LLM. Puedes usar la app como punto de partida para tu propia app para iOS o consultarla cuando modifiques una app existente. El código de ejemplo se aloja en [GitHub](https://github.com/google-ai-edge/mediapipe-samples/tree/main/examples/llm_inference/ios) (https://github.com/google-ai-edge/mediapipe-samples/tree/main/examples/llm_inference/ios).

Clona el repositorio de git con el siguiente comando:

```
git clone https://github.com/google-ai-edge/mediapipe-samples
```

Después de crear una versión local del código de ejemplo, puedes importar el proyecto a iOS Studio y ejecutar la app. Para obtener más información, consulta la [Guía de configuración para iOS](https://ai.google.dev/mediapipe/solutions/setup_ios?hl=es-419) (https://ai.google.dev/mediapipe/solutions/setup_ios?hl=es-419).

Opciones de configuración

Usa las siguientes opciones de configuración para configurar una app para iOS:

Nombre de la opción	Descripción	Rango de valores	Valor predeterminado
modelPath	Es la ruta de acceso a la ubicación en la que se almacena el modelo dentro del directorio del proyecto.	PATH	N/A
maxTokens	Es la cantidad máxima de tokens (tokens de entrada + tokens de salida) que controla el modelo.	Número entero	512
topk	Es la cantidad de tokens que considera el modelo en cada paso de generación. Limita las predicciones a los tokens más probables de Top-K.	Número entero	40
temperature	Es la cantidad de aleatoriedad que se introduce durante la generación. Una temperatura más alta genera más creatividad en el texto generado, mientras que una temperatura más baja produce una generación más predecible.	Número de punto flotante	0.8

Nombre de la opción	Descripción	Rango de valores	Valor predeterminado
<code>randomSeed</code>	Es la semilla aleatoria que se usa durante la generación de texto.	Número entero	0
<code>loraPath</code>	Es la ruta de acceso absoluta al modelo de LoRA de forma local en el dispositivo. Nota: Solo es compatible con modelos de GPU.	PATH	N/A

Conversión de modelos

La API de inferencia de LLM es compatible con los siguientes tipos de modelos, algunos de los cuales requieren conversión de modelos. Usa la tabla para identificar el método de pasos necesarios para tu modelo.

Modelos	Método de conversión	Plataformas compatibles	File type
Gemma-3 1B	No se requiere una conversión	Android y Web	.task
Gemma 2B, Gemma 7B y Gemma 2 2B	No se requiere una conversión	Android, iOS y Web	.bin
Phi-2, StableLM y Falcon	Secuencia de comandos de conversión de MediaPipe (https://ai.google.dev/edge/mediapipe/solutions/genai/llm_inference/index?hl=es-419#supported-models)	Android, iOS y Web	.bin
Todos los modelos de LLM de PyTorch	Biblioteca generativa de AI Edge Torch (https://ai.google.dev/edge/mediapipe/solutions/genai/llm_inference/index?hl=es-419#pytorch-models)	Android, iOS	.task

Para obtener información sobre cómo convertir otros modelos, consulta la sección [Conversión de modelos](#)

(https://ai.google.dev/edge/mediapipe/solutions/genai/llm_inference/index?hl=es-419#convert-model).

Personalización de LoRA

La API de Inference de LLM admite el ajuste de LoRA (adaptación de clasificación baja) con la biblioteca [PEFT](https://huggingface.co/docs/peft/main/en/index) (<https://huggingface.co/docs/peft/main/en/index>) (ajuste fino eficiente en

parámetros). El ajuste de LoRA personaliza el comportamiento de los LLM a través de un proceso de entrenamiento rentable, lo que crea un pequeño conjunto de pesos entrenables basados en datos de entrenamiento nuevos en lugar de volver a entrenar todo el modelo.

La API de inferencia de LLM admite la adición de ponderaciones de LoRA a las capas de atención de los modelos Gemma-2 2B (<https://huggingface.co/google/gemma-2-2b>), Gemma 2B (<https://huggingface.co/google/gemma-2b>) y Phi-2 (<https://huggingface.co/microsoft/phi-2>). Descarga el modelo en formato `safetensors`.

El modelo de base debe estar en formato `safetensors` para crear pesos de LoRA. Después del entrenamiento de LoRA, puedes convertir los modelos al formato FlatBuffers para ejecutarlos en MediaPipe.

Prepara los pesos de LoRA

Usa la guía de métodos LoRA

(https://huggingface.co/docs/peft/main/en/task_guides/lora_based_methods) de PEFT para entrenar un modelo LoRA ajustado en tu propio conjunto de datos.

La API de LLM Inference solo admite LoRA en las capas de atención, por lo que solo debes especificar las capas de atención en `LoraConfig`:

```
# For Gemma
from peft import LoraConfig
config = LoraConfig(
    r=LORA_RANK,
    target_modules=["q_proj", "v_proj", "k_proj", "o_proj"],
)

# For Phi-2
config = LoraConfig(
    r=LORA_RANK,
    target_modules=["q_proj", "v_proj", "k_proj", "dense"],
)
```

Después de entrenar el conjunto de datos preparado y guardar el modelo, los pesos del modelo LoRA ajustados están disponibles en `adapter_model.safetensors`. El archivo `safetensors` es el punto de control de LoRA que se usa durante la conversión de modelos.

Conversión de modelos

Usa el paquete de Python de MediaPipe para convertir las ponderaciones del modelo en el formato Flatbuffer. `ConversionConfig` especifica las opciones del modelo base junto con las opciones adicionales de LoRA.

Nota: Dado que la API solo admite la inferencia de LoRA con GPU, el backend se debe configurar en 'gpu'.

```
import mediapipe as mp
from mediapipe.tasks.python.genai import converter

config = converter.ConversionConfig(
    # Other params related to base model
    ...
    # Must use gpu backend for LoRA conversion
    backend='gpu',
    # LoRA related params
    lora_ckpt=LORA_CKPT ,
    lora_rank=LORA_RANK ,
    lora_output_tflite_file=LORA_OUTPUT_FILE ,
)

converter.convert_checkpoint(config)
```

El convertidor producirá dos archivos Flatbuffer, uno para el modelo base y otro para el modelo LoRA.

Inferencia de modelos de LoRA

iOS admite LoRa estático durante la inicialización. Para cargar un modelo de LoRA, especifica la ruta de acceso del modelo de LoRA y el LLM base.

```
import MediaPipeTasksGenai

let modelPath = Bundle.main.path(forResource: "model",
                                  ofType: "bin")
let loraPath = Bundle.main.path(forResource: "lora_model",
                                 ofType: "bin")

let options = LlmInferenceOptions()
options.modelPath = modelPath
options.maxTokens = 1000
options.topk = 40
options.temperature = 0.8
options.randomSeed = 101
options.loraPath = loraPath

let llmInference = try LlmInference(options: options)
```

Para ejecutar la inferencia de LLM con LoRA, usa los mismos métodos `generateResponse()` o `generateResponseAsync()` que el modelo base.

Salvo que se indique lo contrario, el contenido de esta página está sujeto a la [licencia Atribución 4.0 de Creative Commons](https://creativecommons.org/licenses/by/4.0/) (<https://creativecommons.org/licenses/by/4.0/>), y los ejemplos de código están sujetos a la [licencia Apache 2.0](https://www.apache.org/licenses/LICENSE-2.0) (<https://www.apache.org/licenses/LICENSE-2.0>). Para obtener más información, consulta las [políticas del sitio de Google Developers](https://developers.google.com/site-policies?hl=es-419) (<https://developers.google.com/site-policies?hl=es-419>). Java es una marca registrada de Oracle o sus afiliados.

Última actualización: 2025-07-24 (UTC)